

Regressão Linear: Probabilidade vs. Otimização

Uma dedução dos pressupostos sobre os erros

Luca WB

2026-08-09

Table of contents

0.1	Breve introdução	2
0.2	Dedução via Otimização	2
0.2.1	O Truque de Notação (Notation Trick)	3
0.2.2	Expandindo a Equação Algébrica	3
0.2.3	A Derivada Matricial (Calculando o Gradiente)	4
0.2.4	Minimização e a Equação Normal	5
0.3	Dedução via Probabilidade/Esperança	5
0.3.1	O pressuposto fundamental	5
0.3.2	Por que assumir $E[u X] = 0$?	6
0.3.3	Ortogonalidade entre erro e variáveis explicativas	7
0.3.4	Da população para a amostra	7
0.4	Comparando as duas deduções	8

0.1 Breve introdução

Esse post surgiu por conta da minha primeira aula da cadeira de Aprendizado Profundo na universidade, nessa aula, o professor fez a demonstração da formula normal da regressão linear. Não era exatamente uma novidade essa formula, eu já sabia demonstrar essa formula de cabeça usando valor esperado condicional, o que me intrigou foi um detalhe da demonstração. Meu professor usou a abordagem de otimização para dedução, calculando o ponto minimo global da função perda/avaliação. Isso me intrigou pelo fato de que, a primeira vista, não tínhamos levantado nenhum pressuposto sobre a natureza do erro do nosso modelo, coisa que temos que fazer para realiar a dedução via valor esperado.

Assim, gastei meu fim de semana seguinte pensando sobre o assunto, pesquisei um pouco na internet sobre a correlção entre as duas abordagens e os pressupostos dela, e não achei nada muito satisfatório, então comecei a rabiscar no papel para ver se esclarecia alguma coisa, e acredito ter achado uma perspectiva interessante para ver a relação entre as duas abordagens no que tange os pressupostos sobre o erro

0.2 Dedução via Otimização

Primeiro, antes de mais nada, é preciso estar familiarizado sobre as deduções, pare que então possamos falar sobre elas.

A ideia central na regressão linear é encontrar os parâmetros (ou pesos) que minimizam o erro das nossas previsões. Assim como no exemplo do gradiente onde olhamos para a derivada para encontrar a direção que reduz o valor da função, aqui o nosso objetivo principal é minimizar a função de custo conhecida como Soma dos Erros Quadráticos (SSE - *Sum of Squared Errors*).

A fórmula clássica do SSE é a soma das diferenças ao quadrado entre o valor real (y_i) e o valor previsto ($\hat{y}_i$):

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

0.2.1 O Truque de Notação (Notation Trick)

Antes de derivarmos, precisamos simplificar nossa notação. Um modelo linear comum tem um viés (intercepto ou *bias*) b e os pesos das características $w_1, \dots, w_m$. A previsão para uma única amostra é escrita como:

$$\hat{y}_i = b + w_1 x_{i1} + \dots + w_m x_{im}$$

Para evitar o trabalho de calcular as derivadas do viés b separadamente dos outros pesos, nós usamos um **truque de notação** (*notation trick*). Nós incorporamos o b no vetor de pesos w (sendo o primeiro elemento, w_0) e adicionamos uma coluna inteira de 1s na nossa matriz de características X .

Dessa forma, a previsão de todas as amostras ao mesmo tempo se transforma em uma elegante multiplicação de matrizes:

$$\hat{y} = Xw$$

Com esse truque, a nossa função de erro SSE pode ser reescrita na sua forma vetorializada:

$$SSE(w) = (y - Xw)^T (y - Xw)$$

0.2.2 Expandindo a Equação Algébrica

Para conseguir derivar a função e encontrar o ponto de mínimo, vamos primeiro expandir a multiplicação dessas matrizes:

$$\begin{aligned} SSE(w) &= (y^T - (Xw)^T)(y - Xw) \\ SSE(w) &= y^T y - y^T Xw - (Xw)^T y + (Xw)^T (Xw) \end{aligned}$$

Aplicando a propriedade da matriz transposta, onde $(AB)^T = B^T A^T$:

$$SSE(w) = y^T y - y^T Xw - w^T X^T y + w^T X^T Xw$$

Aqui entra um detalhe geométrico e algébrico importante, se você observar as dimensões, o termo $y^T Xw$ resulta em um único número, ou seja, é um **escalar**. Como a transposta de um escalar é ele mesmo, podemos dizer que $(y^T Xw)^T = w^T X^T y$. Portanto, os dois termos do meio são idênticos e podemos agrupá-los:

$$SSE(w) = y^T y - 2w^T X^T y + w^T X^T Xw$$

0.2.3 A Derivada Matricial (Calculando o Gradiente)

A ideia básica é que, se queremos minimizar o valor da função, precisamos olhar para a derivada dela. Como SSE é uma função convexa, o seu ponto de mínimo global ocorre exatamente onde a inclinação (o gradiente) é zero. Vamos derivar a função em relação ao vetor de pesos w :

$$\frac{\partial}{\partial w} SSE(w) = \frac{\partial}{\partial w} (y^T y - 2w^T X^T y + w^T X^T Xw)$$

Analisando termo a termo:

1. A derivada de $y^T y$ em relação a w é 0 (pois é apenas uma constante, não depende de w).
2. Usando a regra de derivada matricial $\frac{\partial}{\partial w} (w^T a) = a$, a derivada de $-2w^T X^T y$ é $-2X^T y$.
3. Para formas quadráticas, a regra é $\frac{\partial}{\partial w} (w^T A w) = 2Aw$. Logo, a derivada de $w^T X^T Xw$ é $2X^T Xw$.

Juntando todas as peças, o nosso gradiente é:

$$\frac{\partial}{\partial w} SSE(w) = -2X^T y + 2X^T Xw$$

0.2.4 Minimização e a Equação Normal

Agora, basta aplicar a ideia fundamental da otimização: igualar o gradiente a zero para encontrar o fundo do “vale” da função de custo.

$$\begin{aligned}0 &= -2X^T y + 2X^T X w \\2X^T X w &= 2X^T y\end{aligned}$$

Dividindo os dois lados por 2, temos:

$$X^T X w = X^T y$$

Finalmente, para isolar o nosso vetor de pesos ótimos w , multiplicamos ambos os lados da equação pela inversa da matriz $(X^T X)$:

$$w = (X^T X)^{-1} X^T y$$

E esta é a **Equação Normal** da regressão linear. Ela nos dá, de forma direta e analítica, os pesos exatos que minimizam o erro.

0.3 Dedução via Probabilidade/Esperança

Na seção anterior obtivemos a Equação Normal resolvendo um problema de otimização. Agora vamos chegar ao mesmo resultado partindo de uma interpretação probabilística do modelo linear.

0.3.1 O pressuposto fundamental

Considere o modelo

$$y = X\beta + u,$$

onde u é nosso erro, isto é, representa tudo aquilo que não está sendo explicado pelas variáveis observadas em X (Já estamos usando o trick notion e notação matricial). Por definição do nosso modelo $u = y - X\beta$

A hipótese mais importante é que iremos assumir, é:

$$\boxed{E[u \mid X] = 0}.$$

Em palavras:

Depois de observarmos os valores de X , o erro continua tendo média zero.

Essa hipótese é mais forte do que simplesmente assumir

$$E[u] = 0.$$

De fato,

$$E[u \mid X] = 0 \implies E[u] = 0,$$

mas a recíproca não é verdadeira.

0.3.2 Por que assumir $E[u \mid X] = 0$?

A ideia é que toda a estrutura sistemática explicável por X já foi incorporada ao termo $X\beta$.

Se a média do erro ainda dependesse de X , então existiria informação em X que o modelo não está utilizando.

Por exemplo, suponha que

$$E[u \mid X = x] = x^2.$$

Nesse caso, conhecer x permite prever o erro em média, indicando que o modelo ainda está omitindo uma relação importante.

A condição

$$E[u \mid X] = 0$$

formaliza exatamente a ideia de que o erro contém apenas a parte não explicada do fenômeno.

0.3.3 Ortogonalidade entre erro e variáveis explicativas

Da propriedade da esperança condicional segue que

$$E[Xu] = 0.$$

Isso é verdade, pois como ambos são independentes, podemos escrever como $E[X]E[u]$, e como $E[u] = 0$, temos a formula acima

Substituindo

$$u = y - X\beta,$$

obtemos

$$E[X(y - X\beta)] = 0.$$

Expandindo:

$$E[Xy] - E[XX^T]\beta = 0.$$

Logo,

$$\boxed{E[XX^T]\beta = E[Xy]}$$

Esta é a versão **populacional** da Equação Normal.

0.3.4 Da população para a amostra

Até aqui trabalhamos com quantidades populacionais, isto é, esperanças teóricas que normalmente não conhecemos.

Na prática observamos apenas uma amostra de tamanho n . Assim, substituímos os momentos populacionais pelos seus análogos amostrais:

$$E[Xy] \approx \frac{1}{n}X^Ty,$$

e

$$E[XX^T] \approx \frac{1}{n}X^TX.$$

Substituindo na equação anterior:

$$\frac{1}{n}X^T X \hat{\beta} = \frac{1}{n}X^T y.$$

Multiplicando ambos os lados por n :

$$X^T X \hat{\beta} = X^T y.$$

Finalmente,

$$\boxed{\hat{\beta} = (X^T X)^{-1} X^T y.}$$

0.4 Comparando as duas deduções

Bom, como viram, na dedução via esperança assumimos explicitamente algo sobre o erro, que ele deve ser independente das variáveis preditoras. Ao passo de que a princípio não assumimos nada sobre o erro quando deduzimos via otimização. Entretanto isso não é exatamente verdade

A ideia fundamental aqui é que Otimizar o Erro Quadrático (Deep Learning) e Garantir a Premissa de Ortogonalidade (Estatística) são duas faces da mesma moeda. Quando pedimos para um otimizador (como o Gradient Descent) minimizar a distância entre a previsão e o valor real, ele faz isso calculando derivadas. A derivada do erro quadrático (SSE) revela matematicamente a nossa premissa estatística: o gradiente é literalmente a medida de “quanto o erro ainda está correlacionado com os dados ($X^T u$)”. Quando o gradiente chega a zero (o ponto de mínimo da função de custo), significa que o erro se tornou perfeitamente ortogonal (90 graus) em relação aos dados. Vamos materializar isso com código. No exemplo abaixo, criamos um espaço geométrico simples onde X é o nosso “chão” (dados) e y é o alvo flutuando no espaço. O código prova duas coisas simultaneamente: Geometricamente, o menor erro possível forma um ângulo de 90° com os dados. Na otimização, o gradiente empurra o valor de $X^T u$ exatamente para 0.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1. Definindo o Universo 3D
5 # X é uma matriz 3x2. Suas duas colunas geram um plano 2D no espaço 3D (o "chão" dos dados)
6 X = np.array([[2.0, -1.0],
7               [1.0,  2.0],
8               [0.5,  1.0]])
```



```

9
10 # y é o vetor de respostas reais flutuando solto no espaço 3D (fora do plano X)
11 y = np.array([3.0, 4.0, 5.0])
12
13 # 2. A Visão Estatística: Fórmula Normal (Encontrando a Projeção)
14 #  $w = (X^T X)^{-1} X^T y$ 
15 w_opt = np.linalg.inv(X.T @ X) @ X.T @ y
16
17 # y_hat é a projeção de y no plano (a melhor previsão)
18 y_hat = X @ w_opt
19
20 # O vetor de erro u (a distância que falta de y_hat para chegar em y)
21 erro_opt = y - y_hat
22
23 # ===== PLOTAGEM 3D =====
24 fig = plt.figure(figsize=(10, 8))
25 ax = fig.add_subplot(111, projection='3d')
26
27 # Desenhando o Plano gerado pelas colunas de X (O nosso "Espaço de Hipóteses")
28 s, t = np.meshgrid(np.linspace(-1, 3, 10), np.linspace(-1, 3, 10))
29 X_plane = s * X[0, 0] + t * X[0, 1]
30 Y_plane = s * X[1, 0] + t * X[1, 1]
31 Z_plane = s * X[2, 0] + t * X[2, 1]
32 ax.plot_surface(X_plane, Y_plane, Z_plane, alpha=0.3, color='cyan', edgecolor='none')
33
34 # Função auxiliar para plotar vetores a partir da origem
35 def plot_vector(ax, vec, origin=[0,0,0], color='black', label=''):
36     ax.quiver(origin[0], origin[1], origin[2],
37              vec[0], vec[1], vec[2],
38              color=color, arrow_length_ratio=0.1, label=label)
39
40 # Plotando os vetores no espaço
41 plot_vector(ax, X[:, 0], color='blue', label='Vetor X1 (Base do plano)')
42 plot_vector(ax, X[:, 1], color='blue', label='Vetor X2 (Base do plano)')
43 plot_vector(ax, y, color='green', label='Real y (Alvo flutuando)')
44 plot_vector(ax, y_hat, color='purple', label='Previsão y_hat (Sombra no plano)')
45
46 # Plotando o ERRO (Saindo de y_hat e subindo até y)
47 # Este é o vetor que, por definição ( $X^T u = 0$ ), cai a 90 graus em relação ao plano azul
48 plot_vector(ax, erro_opt, origin=y_hat, color='red', label='Erro u (Cai a 90° do plano)')
49
50 # Adicionando uma linha tracejada para destacar a ortogonalidade
51 ax.plot([y_hat[0], y[0]], [y_hat[1], y[1]], [y_hat[2], y[2]], color='red', linestyle='--')
52
53 # Configurações visuais do gráfico

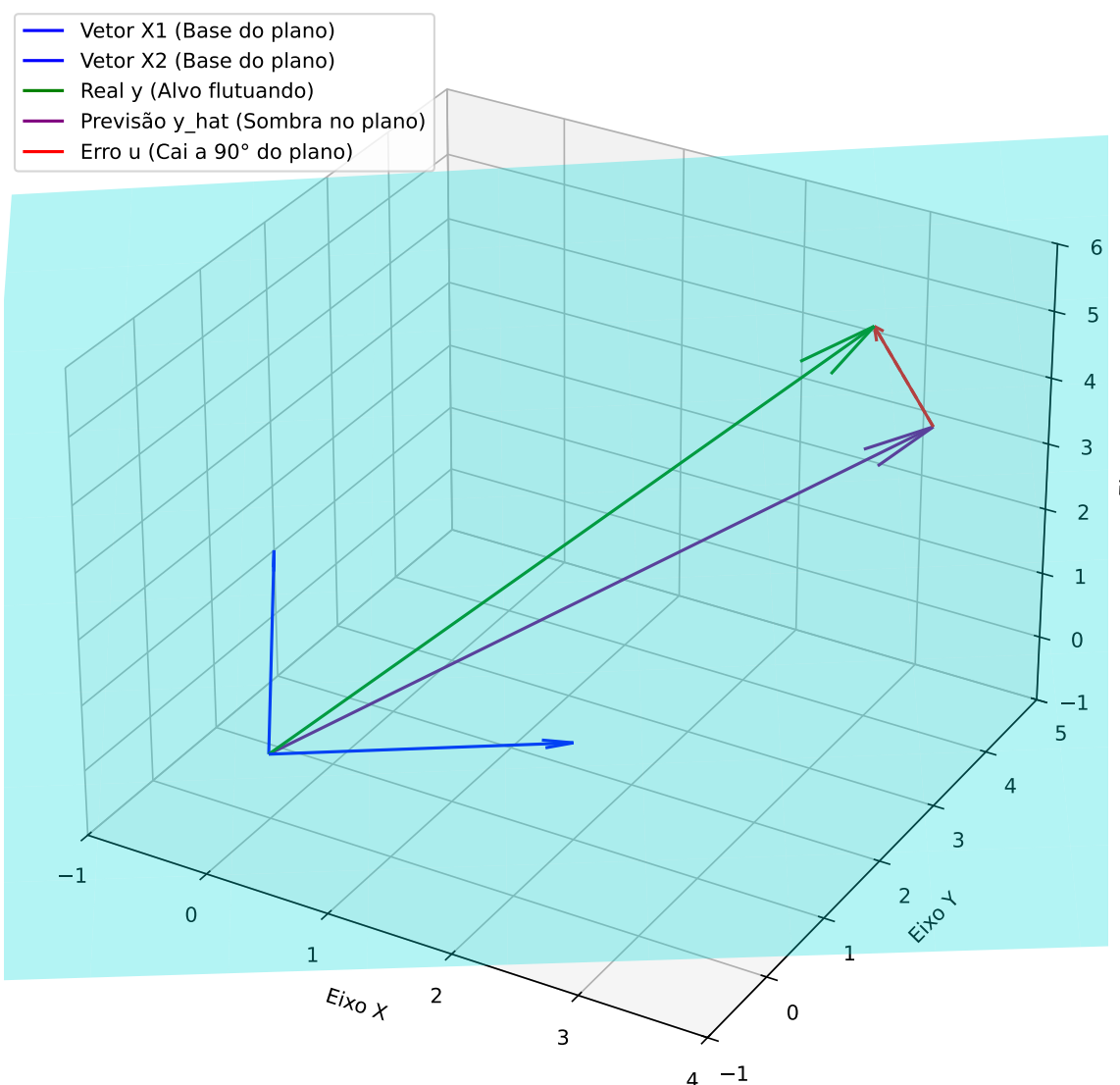
```

```

54 ax.set_xlim([-1, 4])
55 ax.set_ylim([-1, 5])
56 ax.set_zlim([-1, 6])
57 ax.set_xlabel('Eixo X')
58 ax.set_ylabel('Eixo Y')
59 ax.set_zlabel('Eixo Z')
60 ax.set_title('A Geometria do Mínimos Quadrados: O Erro é a Projeção Ortogonal (90°)')
61 ax.legend(loc='upper left')
62
63 plt.tight_layout()
64 plt.show()
65
66 # Verificando a premissa matematicamente
67 print("Prova de que o erro é ortogonal aos dados ( $X^T u = 0$ ):")
68 print(f"Produto interno com X1: {np.dot(X[:, 0], erro_opt):.5f}")
69 print(f"Produto interno com X2: {np.dot(X[:, 1], erro_opt):.5f}")

```

A Geometria do Mínimos Quadrados: O Erro é a Projeção Ortogonal (90°)



Prova de que o erro é ortogonal aos dados ($X^T u = 0$):

Produto interno com X1: -0.00000

Produto interno com X2: 0.00000

O que essa visualização mostra: Gráfico 1 (Esquerda): O vetor vermelho representa o seu erro u . Note que, para que ele seja o menor erro possível (minimização do SSE), ele precisa obrigatoriamente formar um ângulo reto (90 graus) com o vetor azul (seus dados X). Isso é a representação visual da sua equação $X^T u = 0$. Gráfico 2 (Direita): Aqui vemos o que acontece “por debaixo dos panos” numa rede neural. O eixo Y não é a função de custo em si, mas sim o valor da expressão $X^T u$. Conforme o otimizador avança, ele esmaga o valor dessa expressão contra o zero. Em resumo, quando um algoritmo de Deep Learning termina

de treinar usando SSE, ele pode não saber que está fazendo estatística, mas a matemática do gradiente o obriga a cumprir a sua premissa da Folha 1.